LD13 - Block-grouped packed-field reads for sparse contour extraction

Status: implemented in `mdstats 0.19.70a0`
Scope: local-sparse scalar-field access and tiled isosurface extraction
Scientific result: unchanged

1. Objective

Eliminate a pathological random-access pattern in the packed local-sparse density field that made high-resolution tiled contour extraction scale with the number of queried nodes times the number of occupied nodes in a storage block.

The scientific density estimator, logical grid, Gaussian bandwidth, HDR threshold, marching-cubes method, periodic seam treatment, and final mesh are unchanged.

2. Failure mode

A render tile owns at most `32 x 32 x 32` logical cells and gathers a positive-face node halo, normally producing a `33 x 33 x 33` scalar brick:

$$Q = 33^3 = 35,937$$

node queries.

`PeriodicPackedBlockScalarField3D` stores one occupancy bitset and one packed value segment per active storage block. The previous `gather_node_values()` implementation processed every query independently. For every active query node it:

1. found the owning storage block;
2. decoded the complete occupancy bitset for that block;
3. searched the decoded local indices;
4. fetched one packed value.

If a queried block contains P_b positive nodes, repeated queries in that block cost approximately

$$O(Q_b P_b)$$

Python-level bit operations. Gaussian support blocks are commonly almost full, so P_b can approach the complete block-node count.

This storage-efficient but access-inefficient path dominated isosurface rendering. Marching cubes and the final 10–100 thousand triangles were not the primary cost.

3. Public contract

The existing method remains the public access port:

```
def gather_node_values(
    self,
    logical_indices: NDArray[np.integer],
) -> NDArray[np.float64]:
    ...
```

Input

- shape `(n, 3)`;
- integer logical node indices;
- indices may lie outside the canonical cell and are reduced periodically.

Output

- shape `(n,)`;
- `float64` values;
- exact zero for inactive or zero-valued nodes;
- read-only contiguous result;
- query order and duplicate queries are preserved.

No caller-visible API change is introduced.

4. Block-grouped algorithm

For all queries:

1. reduce logical coordinates modulo the logical grid shape;
2. compute storage-block coordinates and local coordinates vectorially;
3. map queried blocks to active packed-block rows with `numpy.searchsorted`;
4. discard inactive-block queries;
5. stable-sort matched query rows by packed-block row;
6. for each distinct touched active block:
   - decode its occupancy bitset once;
   - vectorially search all requested local indices in that block;
   - gather all matching packed values in one operation;
7. scatter values back to their original query rows.

The number of occupancy decodes is therefore

$$N_{\text{decode}} = N_{\text{distinct touched active blocks}},$$

not the number of queried active nodes.

The dominant packed-access complexity becomes approximately

$$O(Q \log B + \sum_{b \in T} P_b),$$

where B is the active-block count and T is the set of touched active blocks.

5. Tiled contour integration

`density_tiled_mesh._gather_tile_volume()` continues to construct the same periodic logical-node queries and call the field access port. It automatically benefits from block-grouped packed-field reads; dense-block and dense-field implementations retain their existing vectorized paths.

The contour pipeline remains:

1. exact HDR candidate-cell identification;
2. deterministic render-tile planning;
3. periodic scalar-brick gathering;
4. one Lewiner marching-cubes call per tile;
5. exact shared logical-edge reconstruction;
6. canonical clipping and periodic seam validation;
7. optional periodic mesh simplification.

6. Resource and timeout semantics

This change reduces execution time but does not weaken resource limits. Worker memory, thread, and wall-time budgets remain authoritative.

Progress completion records now include:

- contour tile count;
- candidate-cell count;
- raw face count;
- final face count;
- shell wall time.

These quantities distinguish scalar-brick access, raw contour work, and final browser geometry.

7. Edge cases

The implementation must preserve:

- empty query arrays;
- all-inactive queries;
- duplicate query indices;
- negative and over-cell periodic indices;
- partial terminal storage blocks;
- incomplete occupancy bitsets;
- sorted packed-value offsets;
- deterministic result order;
- exact equivalence to the sparse reference field.

8. Validation

Focused tests verify:

- packed/reference value equivalence;
- periodic wrapping;
- JSON and HDR behavior;
- one occupancy decode per distinct touched active block;
- tiled and legacy contour equivalence;
- periodic face, edge, and corner seams;
- tile-shape independence;
- raw-workspace admission.

A diagnostic benchmark on the validation runtime used a fully occupied `33^3` tile backed by `16^3` packed storage blocks:

Access path	Time
Historical per-node bitset decode	25.7809 s
LD13 block-grouped decode	0.02135 s
Speedup	1207x

The benchmark isolates packed scalar-brick gathering; end-to-end mesh speedup depends on tile count, raw geometry, topology validation, simplification, serialization, and hardware.

9. Follow-on optimization

After LD13, remaining high-value rendering optimizations are:

1. gather one tile brick once and extract all requested HDR levels for that field;
2. serialize or memory-map one field once per worker rather than once per shell;

3. stream child-worker tile progress through the package `ProgressPort`;
4. optionally parallelize independent tiles when memory and deterministic merge policy permit.

These are secondary to LD13 because the repeated per-node bitset decode was the dominant observed cost.